

Rendu Individuel — Denilsson MOREIRA PEREIRA | Mastère DevOps SUP DE VINCI 2025–2026

Mise en place d'une plateforme CI/CD et d'un environnement DevOps industrialisé

Mastère DevOps — Projet d'études 2025 - 2026

MOREIRA PEREIRA Denilsson Rôle : Responsable Conteneurs & Orchestration

Sommaire

1. [Perspectives d'évolution et réflexion sur l'avenir de la solution](#)
 2. 1.1 [Évolution de l'orchestration vers une architecture multi-cluster](#)
 3. 1.2 [Industrialisation de l'Infrastructure as Code](#)
 4. 1.3 [Renforcement du GitOps et de la gestion des environnements](#)
 5. [Analyse critique — Limites techniques rencontrées](#)
 6. 2.1 [Complexité du bootstrap Kubernetes : un workflow monolithique](#)
 7. 2.2 [Problèmes de pods ArgoCD : dépendances inter-applications non gérées](#)
 8. 2.3 [Helm Charts minimalistes : absence de templating avancé](#)
 9. 2.4 [Persistence désactivée : perte de données au redémarrage](#)
 10. 2.5 [Scaleway Kapsule mono-région : single point of failure](#)
 11. [Annexes](#)
 12. 3.1 [Documentation utilisateur — Guide de prise en main de la plateforme](#)
 13. 3.2 [Analyse personnelle](#)
-

1. Perspectives d'évolution et réflexion sur l'avenir de la solution

1.1 Évolution de l'orchestration vers une architecture multi-cluster

La plateforme DevOps Factory repose actuellement sur un unique cluster Scaleway Kapsule en région fr-par, avec un seul node pool configuré via Terraform. Cette architecture, adaptée au MVP et à la démonstration, présente des limites évidentes en termes de résilience et de haute disponibilité pour un usage en production réelle.

Réplication multi-région

Une évolution majeure consisterait à déployer un second cluster Kapsule dans une région distincte (nl-ams par exemple), couplé à un load balancer anycast pour garantir la continuité de service en cas de panne régionale. Cette architecture permettrait de définir un PRA/PCA documenté avec des objectifs de RPO/RTO mesurables, répondant aux exigences de disponibilité d'une ESN livrant des applications critiques pour ses clients PME.

Scalabilité automatique

Le cluster actuel utilise un node pool avec des limites min/max définies dans les modules Terraform, mais le Cluster Autoscaler Scaleway n'est pas pleinement exploité. Une évolution consisterait à activer l'autoscaling dynamique des nœuds en fonction de la charge réelle, combiné à des HorizontalPodAutoscaler (HPA) configurés sur les services critiques (service-alpha, node-api, react) avec des seuils CPU et mémoire adaptés aux profils de charge observés.

Service Mesh

L'intégration d'un service mesh tel qu'Istio ou Linkerd permettrait de renforcer l'observabilité inter-services (distributed tracing), d'appliquer du mTLS entre les microservices sans modification du code applicatif, et de gérer finement le trafic management (canary deployments, circuit breakers, rate limiting). Le CNI Cilium déjà en place sur le cluster faciliterait cette intégration grâce à sa compatibilité native avec les politiques réseau avancées.

1.2 Industrialisation de l'Infrastructure as Code

Les modules Terraform développés pour le projet (networking, kapsule-cluster, kapsule-pool, container-registry, object-storage-app, IAM, backend) constituent une base modulaire solide. Plusieurs axes d'évolution permettraient d'atteindre un niveau d'industrialisation supérieur.

Versionnement des modules

Publier les modules Terraform dans un registry privé (Terraform Cloud ou un registry Git-based) avec un versionnement sémantique strict. Cela permettrait aux différents environnements (dev, staging, prod) de consommer des versions spécifiques et testées des modules, évitant les régressions lors de modifications.

Tests d'infrastructure

Intégrer Terratest ou le framework de test natif de Terraform pour valider automatiquement le comportement des modules avant leur application. Par exemple, vérifier qu'un cluster Kapsule provisionné expose bien les endpoints attendus, que les network policies sont correctement appliquées, et que les resource quotas limitent effectivement la consommation par namespace.

Drift detection

Mettre en place une détection automatique de drift entre l'état déclaré dans Terraform et l'état réel de l'infrastructure Scaleway. Un workflow GitHub Actions planifié (cron) exécutant `terraform plan` quotidiennement et alertant l'équipe en cas de divergence permettrait de garantir la conformité continue de l'infrastructure.

1.3 Renforcement du GitOps et de la gestion des environnements

L'architecture ArgoCD app-of-apps mise en place gère actuellement un seul environnement (dev) avec 18 applications synchronisées. Pour un usage production multi-clients, plusieurs évolutions sont envisageables.

Multi-environnements GitOps

Étendre la structure ArgoCD pour gérer trois environnements distincts (dev / staging / prod) avec des pipelines de promotion automatisés. Chaque environnement disposerait de son propre dossier de values dans le repository Git, avec des mécanismes de promotion contrôlés (merge request de dev vers staging, puis staging vers prod) validés par des tests d'intégration automatisés.

ApplicationSets

Remplacer les 18 fichiers Application YAML individuels par des ApplicationSets ArgoCD, permettant de générer dynamiquement les applications à partir de templates et de paramètres. Cela réduirait la duplication de configuration et faciliterait l'ajout de nouveaux tenants ou services sans modification manuelle des manifests.

Progressive Delivery

Intégrer Argo Rollouts pour implémenter des stratégies de déploiement avancées (canary, blue-green) sur les applications des tenants. Combiné aux métriques Prometheus déjà collectées, cela permettrait des rollbacks automatiques basés sur des indicateurs de santé (taux d'erreur, latence) plutôt que des interventions manuelles.

2. Analyse critique — Limites techniques rencontrées

2.1 Complexité du bootstrap Kubernetes : un workflow monolithique

Le workflow `bootstrap-k8s.yml`, avec plus de 900 lignes, constitue le point d'entrée pour l'installation complète de la couche plateforme Kubernetes : namespaces, RBAC, ingress-nginx, cert-manager, sealed-secrets, ArgoCD et le root app. Cette approche monolithique, bien que fonctionnelle, présente plusieurs inconvénients.

L'exécution séquentielle des six jobs (bootstrap → ingress-nginx → cert-manager → sealed-secrets → ArgoCD → root-app) crée une chaîne de dépendances fragile : l'échec d'une étape intermédiaire nécessite de relancer l'intégralité du workflow, ou de gérer manuellement les

étapes déjà complétées. En cours de développement, les multiples itérations de debug ont révélé que cette rigidité ralentissait considérablement le cycle de test.

Leçon retenue : pour un futur projet, découper le bootstrap en workflows indépendants et idempotents, chacun responsable d'un composant unique (un workflow pour ingress-nginx, un pour cert-manager, etc.), avec des checks de pré-requis intégrés permettant une exécution partielle sans effet de bord.

2.2 Problèmes de pods ArgoCD : dépendances inter-applications non gérées

L'intégration des 18 applications via ArgoCD a été la source de nombreux problèmes de pods. Le pattern app-of-apps synchronise toutes les applications simultanément lors du déploiement initial, sans mécanisme natif de gestion de l'ordre de démarrage. Cela a provoqué des situations récurrentes :

- **Keycloak** échouait au démarrage car sa base PostgreSQL n'était pas encore prête, générant des CrashLoopBackOff répétés.
- **Metabase** rencontrait le même problème avec sa dépendance MariaDB.
- **Falcosidekick UI** tentait de se connecter à Falco avant que celui-ci n'ait terminé son initialisation.
- Les applications des tenants (service-alpha, node-api, react) échouaient au pull d'image si le container registry n'avait pas encore été peuplé par le pipeline CI/CD.

La résolution a nécessité des ajustements manuels : configuration de retry policies agressives, augmentation des timeouts de health checks, et dans certains cas, des synchronisations manuelles via l'interface ArgoCD pour forcer l'ordre de démarrage.

Leçon retenue : implémenter des Sync Waves ArgoCD (annotations `argocd.argoproj.io/sync-wave`) pour contrôler explicitement l'ordre de déploiement, et utiliser des init containers avec des scripts de wait-for pour vérifier la disponibilité des dépendances avant le démarrage d'une application.

2.3 Helm Charts minimalistes : absence de templating avancé

Les Helm Charts développés pour les trois applications (service-alpha, node-api, react) et Nextcloud sont fonctionnels mais minimalistes. Chaque chart contient uniquement un Deployment, un Service et un Ingress, sans exploiter les fonctionnalités avancées de Helm.

L'absence de helpers (`_helpers.tpl`) pour la génération de labels et de noms standardisés a conduit à une duplication de logique entre les charts. Les tests Helm (`helm test`) n'ont pas été implémentés, ce qui signifie que la validité des templates n'est vérifiée qu'au moment du déploiement réel sur le cluster, augmentant le risque de régressions.

De plus, les security contexts (runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities) ont été configurés correctement mais de manière statique dans chaque chart, sans possibilité de les ajuster via les values. Une approche plus industrielle aurait consisté à créer un chart

library partagé définissant les bonnes pratiques de sécurité par défaut, hérité par tous les charts applicatifs.

2.4 Persistence désactivée : perte de données au redémarrage

Par choix de simplification pour le MVP, la persistence a été désactivée sur plusieurs composants critiques : Grafana (dashboards et datasources), Prometheus (métriques historiques, rétention configurée à 7 jours mais en mémoire), et Vault (mode développement, stockage in-memory). Un redémarrage de pod entraîne la perte de toutes les données non sauvegardées.

Cette limitation est acceptable pour une démonstration, mais inacceptable pour un environnement de production. L'activation de PersistentVolumeClaims (PVC) avec le storage class Scaleway Block Storage aurait résolu ce problème, au prix d'un coût d'infrastructure supplémentaire et d'une complexité accrue dans la gestion des volumes (snapshots, backup, restauration).

2.5 Scaleway Kapsule mono-région : single point of failure

L'ensemble de l'infrastructure est déployé dans la région fr-par de Scaleway, créant un single point of failure géographique. Une panne de la région parisienne (incident réseau, maintenance prolongée) entraînerait une indisponibilité totale de la plateforme.

De plus, le cluster fonctionne avec un seul node pool, ce qui limite la capacité à isoler les workloads par type de charge (compute-intensive vs memory-intensive) ou par niveau de criticité (plateforme vs applications tenant). Une architecture multi-pool avec des taints et tolerations Kubernetes permettrait une meilleure séparation des responsabilités et une gestion plus fine des ressources.

3. Annexes

3.1 Documentation utilisateur — Guide de prise en main de la plateforme

Prérequis

Pour interagir avec la plateforme DevOps Factory, les outils suivants doivent être installés localement :

- **kubectl** — client Kubernetes
- **helm** — gestionnaire de packages Kubernetes
- **terraform** — provisionnement de l'infrastructure Scaleway
- **gh** — CLI GitHub pour les workflows Actions
- Accès aux secrets GitHub (SCW_ACCESS_KEY, SCW_SECRET_KEY, SONAR_TOKEN, etc.)

Ordre de déploiement

Le déploiement de la plateforme suit un ordre strict en cinq étapes :

1. **Bootstrap des buckets Terraform** (workflow `Bootstrap – State Buckets`) — à exécuter une seule fois.
2. **Déploiement de l'infrastructure Scaleway** (workflow `Deploy Infrastructure`, environment = dev).
3. **Bootstrap Kubernetes** : namespaces, RBAC, ingress-nginx, cert-manager, sealed-secrets, ArgoCD, root app (workflow `Bootstrap Kubernetes Platform`, task = bootstrap).
4. **Pipeline CI/CD applicatif** : tests, SonarCloud, build Docker, scan Trivy, push registry, mise à jour des tags GitOps (workflow `Applications – DevSecOps`).
5. **Déploiement du monitoring** : Prometheus, Grafana, Alertmanager (workflow `Deploy Monitoring – Direct Access`).

Accès aux interfaces

Après bootstrap, récupérer l'IP publique de l'ingress-nginx dans le summary du workflow `Bootstrap Kubernetes Platform`. Toutes les URLs suivent le format `http://<service>.<IP>.nip.io`.

Service	URL	Détails
Portail d'accès	<code>http://portal.<IP>.nip.io</code>	Point d'entrée centralisé vers toutes les applications
ArgoCD	<code>http://argocd.<IP>.nip.io</code>	Gestion GitOps, utilisateur : admin
Grafana	<code>http://grafana.<IP>.nip.io</code>	Dashboards monitoring (mot de passe dans le summary du workflow)
Prometheus	<code>http://prometheus.<IP>.nip.io</code>	Métriques brutes et PromQL
Alertmanager	<code>http://alertmanager.<IP>.nip.io</code>	Gestion des alertes
Keycloak	<code>http://keycloak.<IP>.nip.io</code>	IAM, utilisateur : admin / devops-factory-keycloak
Vault	<code>http://vault.<IP>.nip.io</code>	Gestion des secrets (mode dev, non persistant)
Wiki.js	<code>http://wikijs.<IP>.nip.io</code>	Documentation collaborative
Metabase	<code>http://metabase.<IP>.nip.io</code>	Dashboards analytiques
WordPress		CMS client

Service	URL	Détails
	<code>http://wordpress.<IP>.nip.io</code>	
Ghost	<code>http://ghost.<IP>.nip.io</code>	Blog/CMS
Gitea	<code>http://gitea.<IP>.nip.io</code>	Service Git auto-hébergé
Mattermost	<code>http://slack.<IP>.nip.io</code>	Messagerie d'équipe
Nextcloud	<code>http://cloud.<IP>.nip.io</code>	Cloud collaboratif
Flask (service-alpha)	<code>http://flask.<IP>.nip.io</code>	API Python FastAPI
Node API	<code>http://node-api.<IP>.nip.io</code>	API Node.js Express
React	<code>http://react.<IP>.nip.io</code>	Frontend React
Falco	<code>http://falco.<IP>.nip.io</code>	Alertes comportementales runtime (Falcosidekick UI)
Trivy	<code>http://trivy.<IP>.nip.io/metrics</code>	Métriques de scan sécurité

Destruction de l'environnement

Pour éviter des coûts Scaleway inutiles, l'environnement dev se détruit via le workflow `Destroy Infrastructure` (confirm = destroy). Les buckets Terraform state ne sont pas supprimés automatiquement ; ils doivent être retirés manuellement depuis la console Scaleway si une suppression totale est souhaitée.

3.2 Analyse personnelle

Réflexion sur les défis rencontrés

L'ensemble des technologies utilisées dans ce projet — Terraform, Kubernetes, Helm, ArgoCD, Docker, GitHub Actions — m'étaient totalement inconnues au début de la formation. J'ai abordé ce projet en autodidacte, en m'appuyant sur des projets personnels réalisés en parallèle pour monter en compétence avant de contribuer efficacement à l'équipe.

Le premier défi majeur a été la mise en place de l'infrastructure Terraform sur Scaleway. Concevoir des modules réutilisables (networking, kapsule-cluster, kapsule-pool, container-registry, object-storage, backend) en partant de zéro a nécessité de comprendre non seulement la syntaxe HCL, mais aussi l'architecture cloud de Scaleway (VPC, private networks, Kapsule) et les bonnes pratiques d'Infrastructure as Code (state management, backend S3, idempotence).

Le second défi, et probablement le plus formateur, a été la résolution des problèmes de pods sur ArgoCD. Voir des applications en CrashLoopBackOff ou en ImagePullBackOff sans comprendre immédiatement la cause m'a obligé à développer une méthodologie de debug systématique : lecture des events Kubernetes (`kubectl describe pod`), analyse des logs (`kubectl logs`), vérification des dépendances (bases de données, registres d'images), et compréhension des mécanismes de health checks et de readiness probes. Chaque erreur résolue a consolidé ma compréhension de l'écosystème Kubernetes.

La construction des workflows GitHub Actions pour le déploiement a également été une source de frustration productive. Le workflow de bootstrap Kubernetes, avec ses 900+ lignes et ses six jobs séquentiels, a nécessité de nombreuses itérations pour gérer les cas d'erreur, les timeouts, et les dépendances entre composants. Chaque échec de pipeline m'a appris à mieux structurer les workflows et à anticiper les points de défaillance.

Identification des forces et faiblesses personnelles

Forces identifiées au cours du projet :

- **Autonomie et autodidaxie** : capacité à apprendre seul des technologies complexes (Terraform, Kubernetes, ArgoCD) en partant de zéro, en m'appuyant sur la documentation officielle, des tutoriels et des projets personnels.
- **Curiosité technique** : intérêt naturel pour explorer de nouvelles solutions et comprendre le fonctionnement interne des outils, au-delà de la simple utilisation en surface.
- **Persévérance face aux erreurs** : capacité à itérer sur des problèmes complexes (pods en échec, workflows cassés) sans abandonner, en adoptant une approche méthodique de debug.
- **Vision d'ensemble** : capacité à concevoir l'architecture globale de la plateforme (de l'infrastructure Terraform jusqu'au GitOps ArgoCD) en maintenant la cohérence entre les couches.

Axes de faiblesse identifiés :

- **Tendance à vouloir tout faire seul** : difficulté à déléguer certaines tâches, ce qui a parfois conduit à une surcharge de travail personnelle et à un manque de visibilité pour les autres membres de l'équipe sur l'avancement de certains composants.
- **Documentation tardive** : la pression du développement a souvent repoussé la rédaction de la documentation à la fin des sprints, au lieu de la maintenir en continu avec le code.
- **Sous-estimation de la complexité** : tendance à sous-estimer le temps nécessaire pour certaines tâches (notamment le bootstrap Kubernetes et l'intégration ArgoCD), ce qui a impacté le planning initial.

Compétences développées

Compétence	Avant le projet	Après le projet
Terraform (IaC)	Aucune connaissance	Maîtrisé
Kubernetes (manifests, Helm)	Aucune connaissance	Maîtrisé
Docker & conteneurisation	Aucune connaissance	Maîtrisé
ArgoCD & GitOps	Aucune connaissance	Maîtrisé
GitHub Actions (CI/CD)	Aucune connaissance	Maîtrisé
Helm Charts (templating)	Aucune connaissance	Acquis
Scaleway Cloud	Aucune connaissance	Acquis
Network Policies & RBAC	Aucune connaissance	Acquis
Monitoring (Prometheus / Grafana)	Aucune connaissance	En progression
Sécurité applicative (Trivy, Falco)	Aucune connaissance	En progression

Axes d'amélioration personnels pour de futurs projets

- **Améliorer la délégation** : apprendre à faire confiance aux membres de l'équipe pour certaines tâches, en définissant clairement les interfaces et les responsabilités dès le début du projet, plutôt que de centraliser l'ensemble de l'implémentation.
- **Documenter en continu** : adopter une approche de documentation-as-code dès le premier commit, en incluant des README par module Terraform, des commentaires dans les Helm Charts, et des runbooks opérationnels rédigés au fil du développement.
- **Approfondir le testing d'infrastructure** : intégrer des outils comme Terratest, helm unittest, et des tests d'intégration Kubernetes (kind + pytest) pour valider automatiquement le comportement de l'infrastructure avant chaque déploiement.
- **Explorer les architectures multi-cluster** : monter en compétence sur les solutions de fédération Kubernetes (Liqo, Admiralty) et les patterns multi-région pour concevoir des architectures véritablement résilientes dès la phase de design.
- **Renforcer la gestion du temps** : utiliser des techniques d'estimation plus réalistes (planning poker, estimation par analogie) et intégrer systématiquement des marges de sécurité pour les tâches exploratoires sur des technologies non maîtrisées.